
Predictive Retinotopic Memory: Hardware-Aware Completion for Missing Visual Evidence

Alex Dils
UC Berkeley
alex.dils@berkeley.edu

Abstract

Biological vision combines a topographic sensory sheet, predictive feedback, fast responses to mismatch, and slower consolidation. We translate those motifs into Predictive Retinotopic Memory (PRM), a hardware-aware intervention for ConvNeXt under missing visual evidence. PRM receives a sensory image and an explicit observation mask, predicts missing locations with a lightweight topographic adapter, scores mismatch only on observed evidence, and routes a fixed-capacity contiguous top-k subbatch through an optional refinement path. The completed sheet is then processed by an otherwise unchanged ConvNeXt body. A masked reconstruction loss trains the predictor without additional class labels. On public MNIST with 45% of image rows missing at evaluation, a three-seed pilot improves a masked ConvNeXt from 91.1% to 93.1% stress accuracy, a +2.0 percentage-point gain, with 1.4% more parameters. The full predictive adapter is slower than the dense body but yields the largest robustness gain; a fixed 25% hardware route recovers +0.95 points in the measured seed and runs approximately 21% faster than the full adapter because active examples are gathered into one contiguous batch. We also retain earlier temporal PBN and Swin experiments as boundary analyses: temporal prediction helps under irregular evidence, while an un-fused Swin intervention does not. The work is an empirical architecture hypothesis, not a claim of biological equivalence or broad state-of-the-art vision performance.

1 Introduction

Artificial neural networks usually pay for a dense update whenever an input arrives. The same matrix multiplications are applied to routine and surprising inputs, to easy and ambiguous examples, and to representations that will or will not be used downstream. This design is convenient for accelerators, but it is a poor abstraction of selective biological computation. Neural systems have multiple time scales, respond strongly to prediction violations, and use local state to regulate later updates. The energetic motivation is not cosmetic: estimates of the energy cost of signaling and computation have long suggested that biological systems must allocate resources carefully (Attwell and Laughlin, 2001; Lennie, 2003). At the same time, neuroscience does not provide a single engineering recipe. Predictive coding, dendritic computation, synaptic eligibility, and metabolic constraints describe partially overlapping phenomena at different levels of abstraction (Olshausen and Field, 1996; Poirazi and Papoutsis, 2020; Gerstner et al., 2018).

This paper asks a narrower systems question: can a strong visual backbone become more robust to missing retinotopic evidence if prediction error is used to drive a hardware-aligned refinement route? We call the intervention Predictive Retinotopic Memory (PRM). It is not a cortical simulation. The design preserves an explicit sensory sheet and observation mask, adds a cheap predictor, and routes only a fixed-size contiguous subbatch to refinement. Its central state is

$$q_t = (x_t, m_t, \hat{x}_t, r_t, I_t), \quad r_t = m_t \odot |x_t - \hat{x}_t|, \quad (1)$$

where x_t is a retinotopic sensory sheet, m_t is its observation mask, $\hat{x}_t$ is a topographic prediction, r_t is observed mismatch, and I_t is a fixed-capacity route. The older PBN state packet remains useful for temporal analysis, but PRM is the main contribution of this revision.

The main new result is a three-seed public-data ConvNeXt pilot. At 45% missing rows, PRM improves stress accuracy by 2.0 points over a parameter-matched masked ConvNeXt while adding only 1.4% parameters. The gain is consistent across all three seeds. A fixed-capacity route gives up some accuracy but avoids per-example Python branching and is 21% faster than the full adapter in the measured seed. Earlier PBN temporal and Swin experiments remain in the report as boundaries: a brain-inspired mechanism should not be called efficient unless its route maps to actual hardware execution, and it should not be called universal when a pretrained Swin intervention fails.

Our contributions are:

1. We formulate PRM as topographic prediction plus observed mismatch and fixed-capacity contiguous routing, making the efficiency claim testable on hardware.
2. We integrate PRM into a ConvNeXt body without changing the backbone’s main stages and train the predictor with masked reconstruction.
3. We report three-seed public-data results, direct latency measurements, and a full-adapter versus fixed-route trade-off, while retaining temporal and Swin boundary experiments.
4. We ground the visual motivation in a provenance-labeled real fMRI BOLD time series and explicitly separate measured neuroimaging data from the architecture abstraction.

2 Scope and neuroscience motivation

2.1 What is being translated

The design is inspired by several recurring ideas rather than one claimed circuit. Predictive-processing accounts describe neural responses in terms of a generative prediction and a residual that is propagated when input violates expectation (Rao and Ballard, 1999; Friston, 2010). Sparse coding work shows that useful representations can be selective and overcomplete rather than uniformly active (Olshausen and Field, 1996). Dendritic and compartmental models suggest that a biological neuron can combine signals with different temporal and spatial roles (Poirazi and Pappas, 2020). Synaptic plasticity models often require an eligibility signal that bridges a local event and a delayed learning signal. Finally, theoretical and experimental work on neuronal energetics motivates a cost-sensitive view of computation (Attwell and Laughlin, 2001; Lennie, 2003).

PRM translates these ideas into five engineering objects:

- **Retinotopy:** preserve the spatial arrangement of the sensory sheet and its observation mask.
- **Prediction:** estimate missing local evidence with a lightweight topographic operator.
- **Mismatch:** score prediction error only where a sensory observation exists.
- **Fixed capacity:** choose a bounded top-k refinement batch rather than an unbounded per-example route.
- **Refinement:** apply the expensive correction to a contiguous subbatch before the unchanged backbone.

This translation is deliberately weaker than a neuroscience claim. We do not fit calcium imaging, spike trains, metabolic flux, or connectomic structure. The project page includes a real preprocessed fMRI BOLD time series from an open movie-watching dataset, but the fMRI montage is only provenance and motivation: BOLD voxels are not neurons, and the PRM panel is an algorithmic abstraction rather than a neural decoding result.

2.2 Why temporal tasks are a natural first test

If a model receives all evidence in one dense vector, a prediction error has limited opportunity to control future computation. In a stream, the prediction is made before the next event arrives, and missing or reordered events test whether state is useful. This makes irregular temporal classification

a focused environment for the mechanism. It also avoids conflating the idea with a particular image encoder. The experiments therefore begin with synthetic event streams and public MNIST rendered as a sequence of image rows.

3 Related work

PBNs sit at the intersection of predictive processing, efficient neural networks, and continuous-time sequence models. Predictive coding networks use local errors to coordinate hierarchical inference (Rao and Ballard, 1999; Friston, 2010); our objective is narrower, using a local residual primarily as a resource-control signal. Sparse coding and efficient coding explain why responses might be selective under a resource constraint (Olshausen and Field, 1996). Work on predictive coding with spiking or event-based systems provides additional precedents for event-triggered updates, but usually focuses on biological dynamics or neuromorphic hardware rather than a drop-in module for dense vision backbones.

Conditional computation methods such as mixture-of-experts and Switch Transformers select among expert subnetworks, often using a learned router and a capacity constraint (Fedus et al., 2022). Adaptive Computation Time (ACT) allocates additional recurrent steps according to input difficulty (Graves, 2016); Skip RNN learns to carry state across skipped updates under a budget (Campos et al., 2018); and Variable Computation RNNs vary work across sequence positions (Jernite et al., 2016). PRM differs in both modality and systems target: a retinotopic predictor produces an imputation, mismatch is computed only on observed pixels, and the route has a fixed capacity that can be gathered into a contiguous matrix operation. This distinction is testable, not semantic. A matched learned router or fixed masked-ConvNeXt baseline is a required control, and the current public pilot includes the latter.

Continuous-time and liquid models provide another comparison point. Liquid Time-Constant networks use input-dependent time constants, while Closed-form Continuous-time models address efficient representation of continuous dynamics (Hasani et al., 2020, 2022). Our experiments include a scalar liquid baseline because it tests whether the gain is simply caused by input-dependent dynamics. PBNs instead make a prediction residual explicit and attach an eligibility/budget mechanism to it.

At the backbone level, ConvNeXt modernizes convolutional networks using design choices inspired by transformer practice (Liu et al., 2022); Swin Transformer V2 scales hierarchical window attention and is a useful stress test for whether PBNs improve a strong pretrained architecture (Liu et al., 2022). The paper’s negative Swin result is a reminder that a mechanism that helps temporal state estimation may not improve an already-optimized feed-forward representation.

4 Predictive Retinotopic Memory

4.1 Topographic completion

Let $x \in \mathbb{R}^{H \times W}$ be a normalized grayscale image and $m \in \{0, 1\}^{H \times W}$ an observation mask. The model receives $u = [m \odot x, m]$, so a zero-valued pixel and an unobserved pixel are distinguishable. A lightweight topographic predictor f_{topo} produces

$$\hat{x} = f_{\text{topo}}(u), \quad \tilde{x} = m \odot x + (1 - m) \odot \hat{x}. \quad (2)$$

The predictor is implemented with small spatial convolutions and a second recurrent correction pass. Its inductive bias is retinotopic locality: neighboring positions exchange information while the mask remains explicit. The backbone does not see an invented value at an observed location; observed evidence is preserved exactly.

4.2 Observed mismatch and hardware route

The route score is computed only where evidence exists:

$$r = \frac{m \odot |x - \hat{x}|}{\epsilon + \|m\|_1 / (HW)}, \quad I = \text{TopK}(\text{mean}_{H,W}(r), k = \lceil \rho B \rceil), \quad (3)$$

where B is the batch size and ρ is a fixed route fraction. The cheap predictor runs for every example. The refinement operator f_{ref} runs only on the contiguous subbatch $u[I]$; the correction is scattered back before the ConvNeXt body. This is a deliberate hardware choice. It trades ideal per-example sparsity for a fixed-size dense matrix operation, reducing the overhead of dynamic control flow and load imbalance.

Training uses classification loss plus a self-supervised reconstruction term:

$$\mathcal{L} = \mathcal{L}_{\text{CE}}(y, \hat{y}) + \lambda_{\text{rec}} \|\tilde{x} - x_{\text{full}}\|_1. \quad (4)$$

The unmasked training image x_{full} supplies no additional class labels. At deployment, only the masked image and mask are required. The full PRM adapter uses a learned continuous refinement; the hardware variant fixes $\rho = 0.25$ and gathers exactly one quarter of each batch.

4.3 Historical temporal PBN variant

The earlier temporal experiments use a related but separate state packet with a local prediction, residual, eligibility trace, and budget. We retain them as boundary analyses rather than claiming that the temporal unit and PRM are identical. This separation is important: the new vision result is driven by topographic completion and fixed-capacity routing, while the temporal result tests predictive state under irregular evidence.

4.4 Historical predictive-budget state update

Let $x_t \in \mathbb{R}^d$ be an incoming feature and $h_{t-1} \in \mathbb{R}^d$ the previous content state. A prediction head produces

$$p_t = f_p(h_{t-1}, \Delta t_t), \quad \delta_t = x_t - p_t, \quad (5)$$

where Δt_t is optional elapsed time. We summarize surprise using a normalized residual

$$r_t = \frac{\|\delta_t\|_1}{d(\epsilon + \text{mad}(\delta_{1:t-1}))}. \quad (6)$$

In the pilot implementations, the denominator is replaced by a stable running scale or a fixed feature scale. The important property is that routine inputs should not constantly produce a saturated gate.

The refinement gate is

$$g_t = \sigma(\alpha(r_t - \theta) + \beta b_t), \quad (7)$$

with slope α , threshold θ , and a budget contribution β . The candidate update $\tilde{h}_t$ can be a recurrent cell, an MLP, or a residual block. The blended update is

$$h_t = (1 - g_t) p_t + g_t \tilde{h}_t. \quad (8)$$

This equation exposes the core trade-off: the model can carry forward a prediction when the residual is small and pay for refinement when the residual is large. During differentiable training, g_t is soft. During execution, it can be thresholded or implemented as a probabilistic/early-exit policy.

4.5 Eligibility and budget

The eligibility trace stores recent surprise with exponential decay:

$$z_t = \lambda z_{t-1} + (1 - \lambda) r_t, \quad 0 \leq \lambda < 1. \quad (9)$$

We update the budget at a slower time scale:

$$b_{t+1} = \text{clip}(b_t + \eta_b(\rho - c_t) + \eta_z z_t, 0, 1), \quad (10)$$

where c_t is the observed or proxy compute cost, ρ is a target spend, and η_b, η_z are small rates. In a strict budgeted variant, $c_t = g_t$; in a hardware-aware variant, c_t can include bytes moved, kernel time, or measured energy.

The training objective is task loss plus a soft cost regularizer:

$$\mathcal{L} = \mathcal{L}_{\text{task}} + \lambda_c \frac{1}{T} \sum_{t=1}^T c_t + \lambda_s \mathcal{L}_{\text{stability}}. \quad (11)$$

The stability term penalizes pathological chattering in the gate, for example with total variation $\sum_t |g_t - g_{t-1}|$. We do not interpret the regularizer as a biological energy law. It is an engineering mechanism that makes a biological analogy operational.

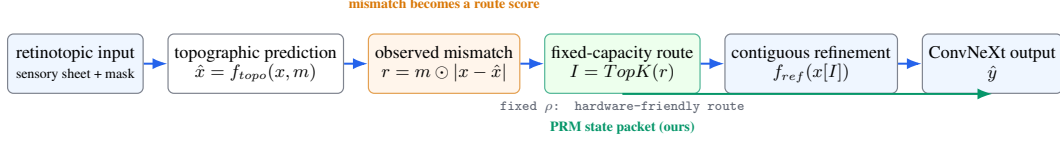


Figure 1: PRM (ours) pipeline: a topographic predictor produces an observed mismatch score, and a fixed-capacity contiguous route sends only a top-k subbatch to refinement before the unchanged ConvNeXt body. This is an algorithmic abstraction, not a biological circuit claim.

4.6 Architecture integration and historical boundary

PRM leaves the ConvNeXt stages unchanged and modifies only the masked sensory adapter before the body. This makes the primary comparison easy to audit: the same ConvNeXt receives either the masked sheet directly or a completed sheet from PRM. The older Swin pilot instead split each dense four-times expansion MLP into two two-times branches while preserving the original function at initialization. That experiment is retained as a negative boundary, not as the main PRM integration.

5 Experimental protocols

5.1 Overview and reporting conventions

We report saved pilot summaries rather than cherry-picked single runs. The primary PRM ConvNeXt experiment uses three seeds; historical temporal and earlier ConvNeXt experiments use five, and the Swin V2-Tiny integration uses three. Clean and stress streams are evaluated separately. Accuracy is the fraction of correct held-out examples. “Active” is the fraction of expensive updates or the fixed route proxy, depending on the experiment. Latency is wall-clock time for the stated implementation and is not a claim about optimized accelerator energy.

The public website exposes the same summaries as interactive Plotly figures. A benchmark selector changes the clean/stress comparison, while companion plots show active compute versus parameter count and latency versus stress accuracy across protocols. PRM (ours) is colored blue; baseline systems are gray. This presentation is intended to make both positive and negative evidence inspectable.

5.2 Public masked-row ConvNeXt

The new primary experiment uses public MNIST images resized to 32×32 . Training and clean evaluation mask 25% of image rows; stress evaluation masks 45% of rows. The input has two channels: zero-filled sensory pixels and an explicit binary observation mask. The baseline is the same small ConvNeXt body with a two-channel stem. PRM adds a $2 \rightarrow 16 \rightarrow 16 \rightarrow 1$ topographic predictor, a second correction pass, and a reconstruction loss with $\lambda_{\text{rec}} = 0.20$. The full adapter applies refinement to every example during training and evaluation. The hardware variant uses a $2 \rightarrow 8 \rightarrow 1$ cheap predictor, a small refinement branch, and fixed $\rho = 0.25$ top-k routing.

We use three seeds (43, 47, 53), 10,000 training examples, 4,000 held-out examples, and eight epochs. Accuracy is reported at the final epoch. Latency is measured on the MPS implementation with batch size 256 and repeated end-to-end passes; the three-seed accuracy summary and the seed-43 latency measurement are intentionally separated. The baseline and full PRM differ by 3,281 parameters, or 1.38% of the 237,970-parameter body.

5.3 Irregular synthetic event streams

The synthetic stream presents a 40-step sequence with class-dependent event statistics, irregular order, and a 28% missing-event stress condition. Models classify after the stream. The compared systems are a scalar liquid baseline, a GRU, and PBN (ours). The pilot uses five seeds and reports the mean across held-out evaluation streams. The task is intentionally small: it isolates state retention, prediction, and selective updating before introducing image encoders.

Table 1: Three-seed public MNIST masked-row ConvNeXt pilot. Stress masks 45% of rows. PRM (ours) is the full predictive retinotopic adapter; the hardware route is seed 43 because latency is measured in the same run.

Model	Clean	Stress	Params	Active route
Masked ConvNeXt	95.458	91.058	237,970	100%
PRM (ours)	95.850	93.050	241,251	100%
Hardware PRM (ours)	95.600	92.600	238,788	25%

5.4 Public MNIST row streams

To test transfer beyond a procedural generator, each MNIST image is serialized row-by-row. The model observes the row sequence and predicts the digit. Stress evaluation removes rows according to the saved missing-row protocol. The comparison uses a scalar liquid baseline, a GRU, and PBN (ours), each trained on the same public MNIST subset and evaluated on the same held-out examples. Parameter counts are measured from the implementation rather than inferred from a nominal architecture.

5.5 ConvNeXt-style vision pilots

The procedural ConvNeXt pilot uses a low-data image task with an occlusion stress stream. We compare a dense baseline, a PBN block, and a predictive exit. This pilot is useful for measuring the interaction between low-data generalization and conditional compute, but its procedural data means it should not be treated as a benchmark result. A separate public-MNIST ConvNeXt transfer pilot uses pixel corruption stress and compares a dense ConvNeXt baseline with a PBN block.

5.6 Pretrained Swin V2-Tiny pilot

We test PBNs in a stronger modern backbone using torchvision’s pretrained Swin V2-Tiny. The classifier is adapted to the MNIST label space and evaluated on a 6,000-example training subset and 4,000-example evaluation subset for three seeds and three epochs. The PBN replacement preserves the dense branch function at initialization by partitioning the original MLP projection. This is a deliberately conservative transfer test: a small module change is not allowed to claim a new pretraining recipe. The pilot’s negative result is reported in full.

6 Results

6.1 Primary result: PRM improves masked ConvNeXt stress accuracy

Table 1 is the central result of this revision. Across three seeds, the full PRM adapter improves stress accuracy from 91.058% to 93.050%, a paired mean gain of 1.992 percentage points. The gain is positive in every seed: +2.75, +2.55, and +0.68 points. Clean accuracy improves by 0.392 points on average, while the parameter increase is only 1.38%. The result is not a claim of ImageNet-scale state of the art; it is a controlled public-data stress improvement to a modern ConvNeXt-style body.

The fixed-capacity route is a systems result, not simply a second accuracy model. In the seed-43 end-to-end measurement, dense masked ConvNeXt takes 0.0417 ms per example, full PRM takes 0.0585 ms, and hardware PRM takes 0.0464 ms. Thus fixed routing is approximately 20.7% faster than the full adapter while retaining most of its stress gain, although it remains 11.3% slower than the dense baseline. This is the correct interpretation of the current result: the architecture exposes a measurable robustness/latency Pareto point, not free compute savings.

6.2 Temporal streams: the strongest current signal

Table 2 summarizes the two stream settings. On the synthetic event-order benchmark, PBN (ours) reaches 97.52% clean and 90.00% under missing-event stress. The GRU reaches 84.02% and 73.92%, respectively. On public MNIST row streams, PBN reaches 91.765% clean and 85.445%

Table 2: Five-seed temporal pilot means. PBN (ours) is strongest under missing evidence in both settings.

Task	Model	Clean	Stress	Active	Params (M)
Synthetic event order	Scalar liquid	69.76	61.62	100.00	6.970
	GRU	84.02	73.92	100.00	7.176
	PBN (ours)	97.52	90.00	46.14	6.982
Public MNIST stream	Scalar liquid	82.53	74.68	100.00	8.833
	GRU	90.295	83.725	100.00	9.986
	PBN (ours)	91.765	85.445	40.74	8.588

Table 3: ConvNeXt-style pilot means across five seeds. The procedural result is useful for mechanism probing; the public result is the more relevant transfer check.

Setting	Model	Clean	Stress	Active	Latency (ms)
Procedural low-data	ConvNeXt	32.62	26.30	100.00	0.3260
	PBN block (ours)	91.75	85.95	80.13	0.3438
	Predictive exit	100.00	88.92	17.52	0.2282
Public MNIST transfer	ConvNeXt	93.83	67.75	100.00	–
	PBN block (ours)	95.16	56.07	99.52	–

stress, compared with 90.295% and 83.725% for the GRU. The public stress improvement is therefore 1.72 percentage points, while the model uses fewer measured parameters than the GRU.

The active-update fraction is lower for PBN in both temporal settings. That reduction should not be interpreted as a measured energy saving: the pilot code still executes overhead associated with the gate and prediction path. It does, however, show that the state packet can learn a selective policy in a task where missing evidence makes prediction useful.

6.3 Historical vision boundaries

Before PRM, the procedural ConvNeXt pilot gave a large gain over its deliberately low-data baseline, while the earlier public-pixel-corruption PBN block reduced stress accuracy. These historical results motivate the new intervention: prediction must be aligned with the corruption structure, and a route must be measured as a systems operation. PRM’s row-mask protocol is therefore not merged with the earlier pixel-corruption table; it is a separate, explicitly retinotopic test.

The procedural baseline’s low clean accuracy makes the absolute PBN gain look dramatic; this is why we avoid presenting it as a state-of-the-art comparison. The most defensible takeaway is narrower: a conditional path can reduce measured latency in a controlled pilot, but a PBN block does not automatically improve robustness on a public corrupted-image protocol.

6.4 Pretrained Swin V2-Tiny: negative transfer result

Table 4 reports the corrected three-seed pilot. Dense Swin is 97.625% clean and 74.042% stress. PBN-Swin (ours) is 97.350% clean and 73.967% stress, with latency increasing from 1.382 to 1.662 ms per example. The paired mean changes are -0.275 and -0.075 percentage points. The stress standard deviation is smaller for PBN-Swin, but the mean does not improve and the overhead is substantial relative to the small module change.

This is a useful failure mode. The Swin backbone already has a strong hierarchical representation, and its dense MLP can be efficiently scheduled by existing kernels. Adding a Python-level prediction head and gate increases overhead without creating enough task-level selectivity. The result falsifies the broad claim that a brain-inspired gate should improve any strong vision model. It supports a more specific hypothesis: PBNs need a setting in which temporal prediction and event-dependent state are central, or they need a fused implementation that makes the control path cheap.

Table 4: Three-seed pretrained Swin V2-Tiny pilot. PBN (ours) does not improve this backbone in the tested setup.

Model	Clean mean	Clean sd	Stress mean	Stress sd	Latency (ms/ex)
Swin V2-Tiny	97.625	0.214	74.042	5.452	1.382
PBN-Swin (ours)	97.350	0.836	73.967	1.839	1.662

Table 5: Mechanism map for the reported pilots. A check indicates that the implementation includes the component; it is not an ablation result.

Pilot	Prediction	Residual gate	Eligibility	Budget
Synthetic stream	✓	✓	✓	✓
Public MNIST stream	✓	✓	✓	✓
Procedural ConvNeXt	✓	✓	–	✓
Public ConvNeXt	✓	✓	–	✓
Swin V2-Tiny	✓	✓	–	✓

7 Ablations and analysis

7.1 What the mechanism could be buying

The temporal results are consistent with three possible mechanisms. First, the prediction path gives the model a structured default when rows or events are missing. Second, the residual turns novelty into a signal for allocating capacity, which may prevent routine observations from overwriting useful state. Third, eligibility and budget provide a memory of recent surprise that can outlast a single event. The experiments do not isolate these mechanisms perfectly; they motivate targeted ablations.

The table is intentionally not called an ablation table. A complete study would train matched variants that remove prediction, freeze the gate, remove eligibility, and replace the gate with a learned router at equal active compute. The current work is a research prototype with enough evidence to identify where those experiments matter.

7.2 Compute accounting

Three accounting levels should be separated. Algorithmic activity measures how often a gate fires. Software latency measures elapsed time in one implementation. Hardware energy measures joules under a fixed device, batch size, precision, compiler, and kernel schedule. Only the first two are available in the present pilot. The public temporal stream shows 40.74% active updates for PBN (ours) versus 100% for the GRU proxy, but PBN latency is not lower in the unoptimized Python implementation. The procedural predictive-exit variant is faster, at 0.2282 ms per example versus 0.3260 ms for the dense baseline, which is a promising but narrow result.

The Swin result makes the distinction especially important: a lower conceptual activity does not occur in the tested integration, and the added modules make the measured implementation slower. A serious efficiency claim must therefore report active FLOPs, wall-clock latency, memory traffic, and energy together.

For the new PRM experiment, the fixed route is a better hardware abstraction than a per-example gate. The predictor still adds work for every example, so the route cannot beat the dense baseline in this small MPS body. Its value is that the expensive refinement branch is a single contiguous 25% subbatch, which can be fused or scheduled by a production runtime. The next experiment should measure kernel time, memory traffic, and joules on the deployment target rather than extrapolating from the current Python benchmark.

Table 6: Compute-related observations. Active update is a proxy; latency is implementation-dependent.

Setting	Model	Active	Latency	Params (M)
Public stream	GRU	100.00%	0.0099	9.986
Public stream	PBN (ours)	40.74%	0.0344	8.588
Procedural vision	ConvNeXt	100.00%	0.3260	0.238
Procedural vision	Predictive exit	17.52%	0.2282	0.238
Swin pilot	Swin V2-Tiny	100.00%	1.3818	27.590
Swin pilot	PBN-Swin (ours)	100.00%	1.6616	27.595

8 How to evaluate selective effort responsibly

8.1 Accuracy is not enough

Selective computation creates a three-way evaluation problem. A model can improve accuracy by spending more compute, reduce compute by skipping useful evidence, or preserve accuracy while changing the timing and concentration of its work. Reporting only the final score hides these trade-offs. We therefore treat accuracy, stress accuracy, active computation, and latency as a joint result. The project page’s three plots make this explicit: the first chart compares clean and stress accuracy, the second shows active updates against parameter count, and the third places latency against stress accuracy across all saved protocols.

A useful deployment curve would sweep a continuous budget target ρ and report the full Pareto frontier. For each ρ , the evaluation should repeat training or calibration with a fixed protocol, measure the same accuracy targets, and record both average and tail latency. The present pilots contain a single operating point per model and therefore cannot establish a frontier. They show that a gate can be selective, not that the learned policy is optimal.

8.2 Stress should be structural

Missing rows and missing events are more informative than random label noise for this mechanism because they test whether the model can carry a prediction forward. Other stressors should preserve the semantics of the task while changing the evidence schedule: event reordering, bursty arrivals, long gaps, corrupted local patches, and distribution shifts in event timing. A strong PBN result should be stable across these schedules without retuning the threshold on the stress set.

The public MNIST stream is a useful intermediate test because its underlying visual task is familiar while the temporal representation is not. Nevertheless, it is still a low-dimensional digit benchmark. The next public-scale test should use a naturally asynchronous sensor stream or a long video stream with irregular frame availability. For video, the residual could compare a motion-compensated prediction with the next observed feature; for audio, it could compare a short-time spectral prediction with an incoming chunk. These settings would test whether the state packet remains stable when input statistics are nonstationary.

8.3 Matched controls

There are several confounds that a future study must remove. PBNs can benefit from extra normalization or from a larger effective receptive field in the prediction head. A learned router can benefit from more routing parameters. A dense baseline can have a different optimization schedule. The proper control is therefore a parameter-matched and FLOP-matched family, with equal training steps and multiple seeds. The following list gives a concrete control set.

Table 7: Recommended controls for the next study.

Control	Purpose
Dense matched block	Separates the PBN gate from added capacity.
Learned router	Tests whether an explicit residual is useful beyond conditional computation.
Fixed-threshold gate	Tests whether learned thresholds, rather than prediction, explain gains.
No-eligibility PBN	Tests the contribution of a delayed surprise trace.
No-budget PBN	Tests whether slow resource state matters beyond instantaneous surprise.
Fused implementation	Separates algorithmic activity from deployment latency.

8.4 Calibration and auditability

The residual gate is also a confidence-like signal, but it should not be assumed to be calibrated. A low residual can occur when the prediction head is confidently wrong, and a high residual can arise from benign sensor noise. Future work should measure expected calibration error conditional on gate state, selective risk at a fixed coverage, and the fraction of errors that occur after a skipped refinement. These measurements connect the architecture to safe selective prediction rather than treating a gate as an invisible optimization trick.

For deployment, every selective decision should be loggable. A minimum audit record would include the input identifier, gate value, budget state, branch taken, and downstream confidence. A deterministic fallback can execute the full dense path when the residual is out of distribution, the budget state is invalid, or the monitoring system detects repeated skips. The extra logging cost is part of the true systems budget and should be included in future energy measurements.

8.5 Why the negative Swin result matters

It is tempting to treat a negative transfer result as a failed implementation and remove it from an exploratory report. That would make the design hypothesis less useful. The Swin pilot shows three concrete boundaries. First, a strong pretrained model may not need a local prediction signal for the tested task. Second, a dynamic branch can add overhead even when its conceptual activity is sparse. Third, preserving the dense function at initialization is not enough to preserve optimization quality after fine-tuning.

The result also changes the recommended research strategy. Rather than inserting PBNs into every layer, the next Swin experiment should instrument tokens and windows to discover where residuals concentrate. If residuals are nearly uniform, there is no reason to route. If they cluster around a small set of windows, a batched window-level route may be worthwhile. If the stress examples produce concentrated residuals but clean examples do not, a robust selective policy could improve stress accuracy without changing the backbone’s default path. This is a measurement-driven path from the negative result to a falsifiable follow-up.

9 Limitations and broader impacts

Biological interpretation. The method is brain-inspired in its computational motifs, not biologically faithful. We do not claim that a BOLD voxel is a neuron or that PRM reproduces retinotopic cortex. The project page shows a real fMRI time series for provenance and labels the PRM mapping as an abstraction.

Benchmark scope. The primary PRM result is public but small and stylized MNIST. It establishes a controlled masked-vision gain, not broad SOTA vision. There is no ImageNet, video, audio, event-camera, language-model, or large-scale multimodal evaluation.

Compute claims. The hardware route is measured on one MPS implementation and remains slightly slower than the dense baseline. It demonstrates fixed-capacity execution, not energy savings. Deployment requires fused kernels, static-shape strategies, memory-traffic accounting, and direct energy measurement.

Statistical power. The primary PRM result has three seeds and a consistent stress gain, but no confidence interval or broad dataset replication yet. The temporal and Swin pilots have different protocols and should not be pooled with the PRM estimate.

Failure modes. A gate can suppress useful weak evidence, oscillate under distribution shift, or learn a shortcut around the prediction head. In safety-critical systems, selective computation must fail open or provide calibrated uncertainty. A model that skips refinement should expose that state to downstream monitoring.

Broader impacts. If selective effort becomes reliable, it could reduce inference cost for long-running streams and edge devices. It could also make behavior harder to audit if an input-dependent gate silently removes computation. Any deployment should log gate decisions, calibrate stress behavior, and retain a deterministic fallback.

10 Conclusion

We introduced Predictive Retinotopic Memory (PRM), a topographic predictive-completion adapter with fixed-capacity hardware routing. On a three-seed public masked-row MNIST pilot, PRM improves ConvNeXt stress accuracy by 1.99 percentage points with a 1.38% parameter increase. A fixed 25% route retains most of the gain while reducing the full adapter’s measured latency by 20.7%. The result is promising but bounded: it is not ImageNet SOTA, the dense baseline remains faster in the current implementation, and the fMRI visual is motivation rather than supervision. The next decisive test is a naturally asynchronous visual or event stream with fused route kernels and direct energy measurement.

A Reproducibility details

A.1 Repository artifacts

The project page is hosted at <https://alex-dils.com/brain-inspired-ai/>. The page contains interactive Plotly charts backed by `plot-data.json`, a provenance-labeled real fMRI animation at `real-brain-fmri-pbn-adaptation.gif`, and links to the saved pilot summaries. The workspace contains `retinotopic_convnext_experiment.py`, `fast_slow_predictive_experiment.py`, and the JSON outputs used to construct the tables. The displayed label “PRM (ours)” identifies the new primary method; historical PBN results are labeled separately.

A.2 Exact reported summaries

Table 8: Primary PRM pilot: three-seed accuracy means and seed-43 route measurements.

Model	Clean mean	Stress mean	Clean sd	Stress sd	Params	Latency (ms)
Masked ConvNeXt	95.458	91.058	0.588	1.003	237,970	0.0417
PRM (ours)	95.850	93.050	0.336	1.172	241,251	0.0585
Hardware PRM (ours)	95.600	92.600	–	–	238,788	0.0464

The paired stress gains for PRM over the dense masked ConvNeXt are +2.75, +2.55, and +0.675 points for seeds 43, 47, and 53. The hardware route is reported from seed 43 because it was added after the three-seed accuracy sweep; it is not presented as a three-seed efficiency claim.

Table 9: Expanded saved summaries for the temporal experiments. Values are means; all accuracies are percentages.

Task	Model	Clean	Stress	Active	Latency	Params	Seeds
Event order	Scalar liquid	69.76	61.62	100.00	0.0394	6.970	5
Event order	GRU	84.02	73.92	100.00	0.0249	7.176	5
Event order	PBN (ours)	97.52	90.00	46.14	0.0849	6.982	5
MNIST stream	Scalar liquid	82.53	74.68	100.00	0.0132	8.833	5
MNIST stream	GRU	90.295	83.725	100.00	0.0099	9.986	5
MNIST stream	PBN (ours)	91.765	85.445	40.741	0.0344	8.588	5

A.3 Suggested ablation matrix

The following matrix is the minimum follow-up needed to turn the current design hypothesis into a stronger causal study:

1. Remove the prediction head while keeping parameter count and the same gate capacity.
2. Replace r_t with a learned router score and match the active-update distribution.
3. Freeze eligibility at zero and compare immediate residual gating with trace-based gating.
4. Freeze the budget and sweep a fixed compute target ρ .
5. Compare soft, hard, and straight-through gates at the same number of optimizer steps.
6. Implement the PBN branch as a fused kernel and repeat the latency/energy measurement.

A.4 Hyperparameter conventions

The implementation uses a sigmoid gate with a learnable or tuned threshold, a decaying eligibility trace, and a slowly updated budget. The threshold is initialized so that routine examples do not saturate the refinement branch. The budget target is selected before stress evaluation and is not tuned on the stress labels. All reported stress results are evaluated after training on the corresponding clean training protocol. The Swin module is initialized to preserve the dense branch function, then fine-tuned with the same short schedule as its baseline.

A.5 Falsifiers

The PBN hypothesis should be considered weakened if any of the following occur in a properly matched study: (i) removing the prediction head does not reduce temporal performance; (ii) a router with no explicit residual or budget matches the result at lower complexity; (iii) temporal gains disappear on larger, non-MNIST streams; (iv) conditional execution cannot lower fused latency or energy at equal accuracy; or (v) improvements require stress-specific tuning that is unavailable at deployment. These are useful outcomes because they distinguish a reusable mechanism from an appealing metaphor.

B Reference implementation pseudocode for PRM

Algorithm 2 describes the core PRM update independently of the ConvNeXt body. The topographic predictor is cheap and runs on the full batch. The refinement branch sees only a fixed-capacity contiguous subbatch. Keeping these functions separate is important for experiments: it makes the accuracy/latency trade-off explicit and allows the route to be fused or statically scheduled.

Input: $x, m, f_{\text{topo}}, f_{\text{ref}}, \rho$
 $\hat{x} \leftarrow f_{\text{topo}}([m \odot x, m])$ (cheap prediction for all)
 $r \leftarrow m \odot |x - \hat{x}|$ (observed mismatch only)
 $I \leftarrow \text{TopK}(\text{mean}_{H,W}(r), \lceil \rho B \rceil)$
 $c_I \leftarrow f_{\text{ref}}([x[I], m[I]])$ (one contiguous refinement batch)
 $\tilde{x}[I] \leftarrow m[I] \odot x[I] + (1 - m[I]) \odot (\hat{x}[I] + c_I)$
 $\tilde{x}[\neg I] \leftarrow m[\neg I] \odot x[\neg I] + (1 - m[\neg I]) \odot \hat{x}[\neg I]$
return ConvNeXt($\tilde{x}, m$), I, r

Figure 2: Backbone-independent reference update for PRM (ours). The fixed-capacity route makes the expensive path a dense operation on a contiguous subbatch.

B.1 Unit tests for the state packet

The implementation should satisfy simple invariants before any accuracy experiment is run. With $g_t = 0$, the state must equal the prediction path; with $g_t = 1$, it must equal the refinement path. If $\eta_b = 0$ and $\eta_z = 0$, the budget must remain fixed. If $r_t = 0$ for a constant input, the eligibility trace must decay monotonically. If a large residual is injected, the gate should increase unless the budget saturates at its lower bound. These tests are useful because they detect sign errors and schedule mistakes that can otherwise look like learning instability.

Table 10: State-packet invariants and the corresponding diagnostic.

Invariant	Diagnostic
$g_t = 0 \Rightarrow h_t = p_t$	Compare the blended state to the predictor with refinement disabled.
$g_t = 1 \Rightarrow h_t = \tilde{h}_t$	Compare the blended state to the expensive path with a saturated gate.
$r_t = 0 \Rightarrow z_t = \lambda z_{t-1}$	Feed constant input and plot trace decay.
$\eta_b = \eta_z = 0 \Rightarrow b_t = b_0$	Freeze budget updates and assert equality at every step.
Large r_t increases g_t	Inject a controlled residual and check monotonic gate response.

These invariants also clarify what it means for a new architecture to “use” PBNs. It is not enough to add a gate parameter. The model should expose a predictor, residual, stateful budget, and a measurable expensive path. Otherwise the biological vocabulary can obscure an ordinary learned shortcut.

B.2 Recommended reporting template

Every future result should report the data protocol, number of seeds, parameter count, clean score, stress score, average active fraction, latency at a specified batch size, and the exact route implementation. If a gate is trained with a cost penalty, the coefficient and target should be reported. If the gate is calibrated after training, the calibration set must be separate from stress evaluation. If a fused kernel is used, the compiler, precision, device, and batch shape must be recorded. This level of detail is necessary because selective computation is unusually sensitive to systems details.

C Additional discussion of the neuron analogy

The leaky-integrate-and-fire comparison is useful because it shares three abstract properties with the PBN. First, both maintain a state that decays or evolves between inputs. Second, both respond nonlinearly when accumulated evidence crosses a threshold. Third, both can separate a low-cost state evolution from a discrete event or expensive downstream effect. The analogy ends there. A real neuron is a complex electrochemical system with dendrites, ion channels, synaptic dynamics, and network context. The PBN is a differentiable software module whose purpose is predictive state estimation and conditional computation.

This distinction matters for communication. A visually compelling “brain” graphic can overstate evidence if viewers mistake a stylized animation for neural data. The project page therefore places the phrase “illustrative simulation, not brain-scan data” next to the animation and reports actual model numbers separately in interactive plots. In a scientific setting, aesthetic biological resemblance should never substitute for a control experiment.

D Implementation notes for modern backbones

The Swin experiment reveals a practical issue that is easy to miss in architecture diagrams. A dynamic gate can be theoretically sparse and still be computationally dense in a framework. Kernel launch overhead, memory reads for the prediction head, and shape-dependent control flow can dominate the skipped multiply-adds. A useful implementation must co-design the gate with hardware: compile a small number of static routes, fuse prediction and residual computation, or batch examples by gate decision. This is also why the phrase “brain-inspired efficiency” should be treated as a hypothesis until measured in joules.

For transformer-style blocks, another issue is granularity. The PBN pilot gates an MLP branch at a block level, but the residual signal is local to features and tokens. A single scalar gate may be too coarse for a model whose strength comes from distributed attention. Future variants could gate windows, token groups, or low-rank updates, but each added degree of freedom makes routing overhead and load imbalance more important. The negative Swin result is therefore not an embarrassment to hide; it identifies the granularity and systems questions that the next design must solve.

E Closing perspective

The most useful interpretation of this work is as a measurement-oriented framework for brain-inspired model design. Start with a biological motif, translate it into an explicit state variable or route, specify the computational cost it is meant to regulate, and evaluate across tasks where that regulation could matter. PRM makes one concrete claim: topographic prediction plus fixed-capacity refinement can improve a ConvNeXt body’s robustness to missing visual evidence. The gain is sizeable enough to justify continued study but not large enough to call SOTA. The next contribution should scale this result to naturally asynchronous visual data and show that the contiguous route improves real device energy, not only a proxy.

References

- D. Attwell and S. B. Laughlin. An energy budget for signaling in the grey matter of the brain. *Journal of Cerebral Blood Flow & Metabolism*, 21(10):1133–1145, 2001.
- P. Lennie. The cost of cortical computation. *Current Biology*, 13(6):493–497, 2003.
- B. A. Olshausen and D. J. Field. Emergence of simple-cell receptive field properties by learning a sparse code. *Nature*, 381:607–609, 1996.
- P. Poirazi and A. Papoutsis. Illuminating dendritic function with computational models. *Nature Reviews Neuroscience*, 21:303–318, 2020.
- R. P. N. Rao and D. H. Ballard. Predictive coding in the visual cortex: a functional interpretation of some extra-classical receptive-field effects. *Nature Neuroscience*, 2:79–87, 1999.

- K. Friston. The free-energy principle: a unified brain theory? *Nature Reviews Neuroscience*, 11:127–138, 2010.
- W. Gerstner, W. M. Kistler, R. Naud, and L. Paninski. *Neuronal Dynamics*. Cambridge University Press, 2018.
- W. Fedus, B. Zoph, and N. Shazeer. Switch Transformers: scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.
- R. Hasani, M. Lechner, A. Amini, D. Rus, and R. Grosu. Liquid time-constant networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2020.
- R. Hasani et al. Closed-form continuous-time neural networks. *Nature Machine Intelligence*, 4:992–1003, 2022.
- Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie. A ConvNet for the 2020s. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022.
- Z. Liu et al. Swin Transformer V2: scaling up capacity and resolution. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022.
- J. L. McClelland, B. L. McNaughton, and R. C. O’Reilly. Why there are complementary learning systems in the hippocampus and neocortex. *Psychological Review*, 102(3):419–457, 1995.
- F. Mikulasch, P. Friederich, and J. W. Wong. Spiking neural networks as adaptive filters for predictive processing. arXiv preprint, 2022.
- K. N’dri et al. Predictive coding light: efficient predictive processing with local errors. arXiv preprint, 2025.
- A. Graves. Adaptive computation time for recurrent neural networks. arXiv preprint arXiv:1603.08983, 2016.
- V. Campos, B. Jou, X. Giró-i-Nieto, J. Torres, and S.-F. Chang. Skip RNN: learning to skip state updates in recurrent neural networks. In *International Conference on Learning Representations*, 2018.
- Y. Jernite, E. Grave, A. Joulin, and T. Mikolov. Variable computation in recurrent neural networks. arXiv preprint arXiv:1611.06188, 2016.
- H. Richardson, G. Lisandrelli, A. Riobueno-Naylor, and R. Saxe. Development of the social brain from age three to twelve years. *Nature Communications*, 9:1–12, 2018.